ACO-Pruning for Deep Neural Networks: A Case Study in CNNs

Renato Sellaro Dorighello*
Universidade Tecnológica Federal do Paraná
Curitiba, Brazil
dorighello@alunos.utfpr.edu.br

Myriam Regattieri Delgado
Universidade Tecnológica Federal do Paraná
Curitiba, Brazil
myriamdelg@utfpr.edu.br

Ricardo Lüders
Universidade Tecnológica Federal do Paraná
Curitiba, Brazil
luders@utfpr.edu.br

Daniel Fernando Pigatto
Universidade Tecnológica Federal do Paraná
Curitiba, Brazil
pigatto@utfpr.edu.br

ABSTRACT

Deep Neural Networks (DNNs) are successful in several tasks, mainly due to their ability to process a large volume of data, given their huge number of parameters and computational operations. Larger and deeper models have been developed to improve their performance with an increasing computational cost. Pruning algorithms are strategies necessary to mitigate the computational burden and achieve better performance by eliminating parts of the network structure while maintaining good training and testing results. Dynamic network pruning increases performance through online choices of inference paths depending on various inputs. This work proposes a new Ant Colony Optimization Pruning (ACO-P) algorithm for dynamic pruning based on swarm intelligence to compress the model without jeopardizing accuracy. We validate ACO-P with a CNN model on the MNIST dataset by comparing it with a baseline pruner that uses random choices, and a well-established dynamic pruning method based on a secondary neural network. The results show that our proposal is a computationally more efficient alternative, capable of achieving higher pruning rates.

CCS CONCEPTS

• **Computing methodologies → Neural networks**.

KEYWORDS

Swarm Intelligence, Pruning Neural Networks, Convolutional Neural Network Optimization.

ACM Reference Format:
Renato Sellaro Dorighello, Myriam Regattieri Delgado, Ricardo Lüders, and Daniel Fernando Pigatto. 2024. ACO-Pruning for Deep Neural Networks: A Case Study in CNNs. In *Genetic and Evolutionary Computation Conference (GECCO '24 Companion), July 14–18, 2024, Melbourne, VIC, Australia.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3638530.3664125

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
GECCO '24 Companion, July 14–18, 2024, Melbourne, VIC, Australia
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0495-6/24/07
https://doi.org/10.1145/3638530.3664125

1 INTRODUCTION

In recent years, research involving Deep Neural Networks (DNNs) has produced increasingly better results within several applications. Some models are enriched with abilities comparable to human capacity. Studies on DNNs have been intensified due to the greater demand for application in diverse areas [7, 10, 14, 39]. Broad and well-established examples are deep models for smart cities, medicine, and even the retail sector for inventory monitoring, especially in e-commerce [3, 12, 34].

Recent studies show that DNNs can be compressed without significantly affecting efficiency, due to the redundancy present in their architectures [6]. Yelding a compressed network, the computational cost can be reduced [27]. Started with shallow models [20], some pruning techniques have been developed to reduce the number of parameters of deeper models [17]. Yet quite efficient, many of these methods consist of removing only redundant network weights [18]. Such a characteristic can increase the network sparsity and might have limited effects on speeding up computational operation [27].

Another important aspect regarding traditional pruning methods is their resulting static model which might be suitable for implementation, but may not align with the evolving landscape of modern DNN models prominent in scientific research, such as Transformers [23], Deep Graph Neural Networks [31], Difusion Models [44] and a focus on generative models [24].

In the context of deep pruning, our proposed approach for DNNs considers a dynamic pruning technique that reduces the computational cost without relevant performance decreasing. More specifically, the present paper aims to explore supporting methods for decision-making (called secondary methods) whose main goal is to choose the parameters of the main network (deep model) that should be compressed. This approach yields an alternative to the secondary network decision algorithm proposed in [28], which shows good image classification results by using a reinforcement learning algorithm for the decision network. The present work aims to evaluate another decision method based on Ant Colony Optimization (ACO) [11].

In the present paper, ACO agents (ants) construct a set of paths within a graph created according to the network structure. The best paths identified by the ACO algorithm are subsequently selected. They have their nodes (representing components of DNNs) removed from the network structure. The proposed approach ACO-Pruning (ACO-P) aims to eliminate redundancies and leverage the generalization process.

In the experiments, results concerning classification accuracy, training time, and pruning rates of CNN models are used to validate the proposed approach. Our pruned models are compared with the original model (ie. without pruning), with a randomly pruned model, and finally with another performance acceleration algorithm based on CNN pruning found in [28]. These methods are applied to the MNIST dataset [26].

It is important to highlight that the proposed approach is general enough to be applied to any deep model composed of simple or complex building block components. The CNN model and MNIST dataset are addressed in a preliminary application of the proposed pruning approach.

Our main contribution is proposing an ACO-based strategy for dynamic pruning that uses gradient maps obtained during DNN training to guide ants toward the most suitable components to be pruned. Although ACO has already been applied to DNN optimization, particularly to Neural Architecture Search (NAS) [1], our proposal differs from previous works because it applies the ACO decision algorithm during the training process. To the best of our knowledge, it is the first time a dynamic pruning strategy based on ACO has been proposed to optimize a CNN structure during its training process. Even general dynamic pruning methods had never directly used ACO to remove weights of artificial neural models.

The paper is organized as follows. Section 2 presents a brief description of the considered techniques (CNN and ACO), and contextualizes the paper concerning the related work of the literature. The fundamentals of the proposed approach are discussed in Section 3 which details the proposed Ant Colony Optimization Pruning (ACO-P) algorithm. Section 4 describes the experiments and presents the results. Conclusions and future directions are discussed in Section 5.

2 BACKGROUND AND RELATED WORK

This section presents some basic aspects of convolutional neural networks followed by a review of relevant papers of the literature.

2.1 Convolutional Neural Networks (CNN)

CNNs are a class of DNNs that use convolution operations to extract features from a data source. CNNs have been most successfully applied to image classification [8, 25, 32]. This task consists of identifying certain elements present in the images, by associating each of its pixels to a certain class (or label), according to the spectral response patterns of those elements.

Figure 1 illustrates a CNN (the complete structure used in our experiments) with three convolutional layers, each followed by a max pooling operation. The network backbone finishes with two fully connected layers, which are used to classify the inputs represented by a handwritten number three in the MNIST dataset.

As illustrated in Figure 1, CNNs use kernels or filters that are smaller than the inputs (either images or feature maps from a previous layer) and go through them jumping from one position to another depending on the stride[1] value, and performing a convolution operation that will result in different output feature maps (one for each kernel). In Figure 1, Padding=1 refers to one extra pixel added around the borders of the layer's input (image or feature map). Padding and stride deal with aggregation bias from the convolution operation by ensuring every pixel is considered.

[1]Stride defines the size of the jump performed in the input, between two sequential operations. Higher stride values provide smaller outputs.

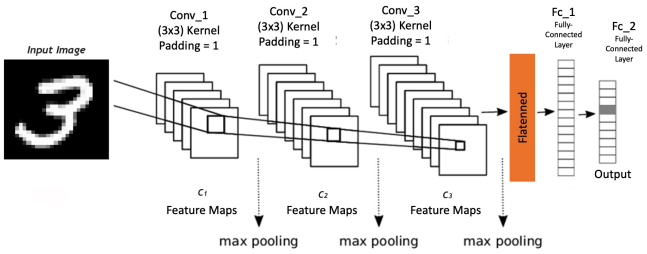

Figure 1: Example of a CNN's backbone with three convolutional layers (each one with several feature maps and followed by max-pooling operations) and two fully connected layers that provide the image classification.

In the CNN depicted in Figure 1, each convolutional layer encompasses C_i different channels formed by a filter (kernel of dimension 3x3 in our case) and its corresponding feature map. The channel is followed by a pooling operation that reduces the feature dimension. As depicted in Figure 1, the two final layers are fully connected. It means they receive 1D inputs (obtained after a flattening process for the first FC layer) and a different weighted connection for each input. When compared with traditional Artificial Neural Networks (ANNs) structures with the same number of layers, CNNs require less memory to store parameters, since there are fewer neural connections, which contributes to greater training efficiency [16] [46]. Sharing parameters is another characteristic that makes CNN less computationally expensive relative to its corresponding ANN. Instead of learning a set of weights for each region of the image, the network learns only one filter, which will be used to detect features from the whole input and form, in the output, the corresponding feature map. Despite being smaller than their ANN counterparts, CNN's deeper structures can result in huge parameters and operation sets.

Figure 2 details one particular layer of the CNN structure shown in Figure 1.

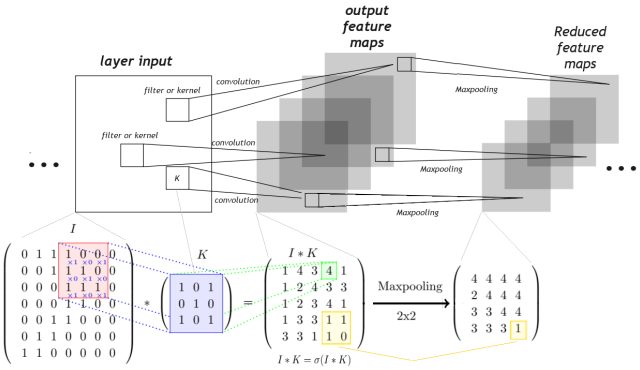

Figure 2: Convolution followed by Maxpooling - an example.

It shows at the top a general view of a particular layer composed of several filters (kernels). Each one yields a distinct feature map, which is then subjected to a non-linear transformation through the activation function σ and then resized through a max-pooling operation. At the bottom, Figure 2 details the convolution operation for a specific kernel (K) at a particular position of the input (I) providing, after the activation function σ, the corresponding output $\sigma(I * K)$. In the example, the rectified linear unit (ReLU), which can introduce non-linearities and solve the vanishing gradients issue, provides the same output ie. $I * K = \sigma(I * K)$ as the input has only non-negative elements. The output of the convolutional layer, followed by the ReLU activation, is subsequently subjected to a 2x2 max-pooling operation. In the example, both operations (convolution and maxpooling) use a unitary stride [29].

2.2 Related Work

Although image classification is the most common CNN application in the literature, there are also successful applications in speech recognition [2, 19], and recommender systems [40]. Time series forecasting [45] is also considered for CNN application, where recurrent models dominate, as well as natural language processing [41], where transformers[2] are the most usual models [4, 30].

CNNs perform well in different tasks, mainly due to their ability to automatically identify features using locally connected layers [25]. Convolution and max-pooling have represented an advance concerning connections per level, nevertheless CNNs have become increasingly larger and deeper [43]. They provide better performance on the one hand, but greater complexity on the other hand. A large amount of parameters in CNNs implies an equally high number of operations and training time. Hence, their utilization in real-time processing applications may pose challenges, particularly when operating within constrained computational resources, such as those encountered in embedded systems. A current example of this type of application is Tiny Machine Learning, whose algorithms are implemented in low-power devices, such as sensors or microcontrollers, to perform automated tasks that involve, for instance, Internet of Things (IoT) [42].

To optimize the use of neural networks in general, and deep models in particular, techniques have been developed in recent years. They aim to reduce neural networks' computational burden by applying different pruning strategies. However, many pruning methods already explored in literature produce a static network model, failing to explore the different properties of newly input data. According to [37], most of these methods yield irregular network structures post-pruning, posing challenges in directly achieving actual computational savings.

In [20], the authors compare biases for minimal network construction using back-propagation. The work explores a method that adds penalty terms to the error function to determine the number of hidden units and enhance generalization ability. Also, [18] presents a method for reducing NN storage and computation requirements, while maintaining accuracy. That method involves training the network to identify important connections, removing redundant ones, and finally, fine-tuning the remaining connections. Another approach is shown in [17], which introduces a three-stage pipeline called "deep compression" to reduce the NN storage requirement without affecting its accuracy. The pipeline consists of pruning, trained quantization, and Huffman coding[3] [22]. Pruning reduces the number of connections, quantization enforces weight sharing, and Huffman coding further compresses the network.

In [28], a pruning approach for a CNN (referred to as the backbone or main network) utilizes a parallel network to manage the pruning of redundant feature maps and their associated kernels using reinforcement learning (RL). This approach introduces a dynamic pruning method for image identification, which adapts the pruning according to the input data. A Deep RL technique, based on the Markov model, acts as a decision-maker to evaluate the importance of each convolutional kernel and prune them accordingly. Consequently, the network undergoes more pruning when an image is easier to classify. For instance, with an input sample of number 3, as shown in Figure 1, and a CNN comprising multiple layers, the approach in [28] automatically determines which filters to prune in each layer. This is achieved through a recurrent network (Recurrent Neural Network - RNN), which models the decision process. Building upon this framework, the current work proposes a more efficient pruning strategy by introducing an ACO-based dynamic pruner that aims to replace the secondary decision network in [28] and supervise the pruning process during CNN training.

ACO is a meta-heuristic for solving combinatorial optimization problems, inspired by observations of the behavior of ant colonies in nature [38]. The ACO algorithm has previously been employed in optimizing network architecture (e.g. NAS) for classification tasks, mainly due to its robust global/local search capabilities and adaptable graph representation [33].

Other examples of ACO applications in NAS are available in the literature. In [38] for instance, an ACO is used for finding the optimal structure of a three-layer ANN; in [5] the Ant Colony System (ACS) algorithm is used for finding the best CNN structure for an image classification problem and [33] proposes a two-stage hybrid ACO for high-dimensional feature selection (TSHFS-ACO). In [1], the authors discuss the usage of meta-heuristics to train and optimize DNNs. The paper provides an overview of various nature-inspired meta-heuristic optimization algorithms that have been applied to train, fine-tune, search for optimal network architectures, and optimize hyperparameters of DNNs. The paper discusses two different applications of ACO techniques. First in [9], they describe an ACO that optimizes a deep recurrent NN structure for general aviation flight data prediction, specifically for airspeed, altitude, and pitch. The second example is [13], which uses ACO to evolve a long short-term memory network for aircraft engine vibrations. In both cases, ACO is used to select the optimal path through the input and hidden layer NN connections, resulting in improved performance compared with other optimization techniques.

As far as we know, the present paper is the first to consider ACO for all pruning decisions. Although other works have already used ACO for NAS, none of them have used ants to directly select which kernels of CNNs should be pruned during the training process.

[2]Composed of encoder and decoder that work together but with several variants already proposed in the literature, transformers can learn to efficiently represent the meaning of a text by analyzing larger bodies of unlabeled data.

[3]Huffman coding reduces data size without losing any information. It assigns shorter binary codes to frequently occurring elemets and longer codes to less frequent ones. This technique allows for efficient data storage and transmission.

3 ACO-P: AN ACO-BASED PRUNING FOR DEEP NEURAL NETWORK OPTIMIZATION

Inspired by the rationale behind the approach presented in [28], our work also proposes a strategy that prunes the network structure during the training process. However, different from [28], which associates an RNN with the Q-learning algorithm, in our case, an ACO-based strategy makes all pruning decisions. In our Ant Colony Optimization Pruning (ACO-P), a brand-new pruning algorithm, ants select subgraphs whose nodes are associated with components most suitable to be removed from the network structure.

Aimed at compressing the model and increasing its generalization, the pruning procedure occurs during the training phase (weights update). ACO-P considers batch gradients associated with a particular component as the heuristic information. It uses the whole path's heuristic in the mechanism of pheromone update.

Figure 3 illustrates the general view of ACO-P, focused on its graph structure. The shown graph $\mathcal{G}$ could be derived, for example, from the CNN with three convolutional layers depicted in Figure 1. In the graph $\mathcal{G}$, each node called *channel* could represent a kernel and its output feature map(s) as detailed in the zoomed channel c depicted in Figure 3.

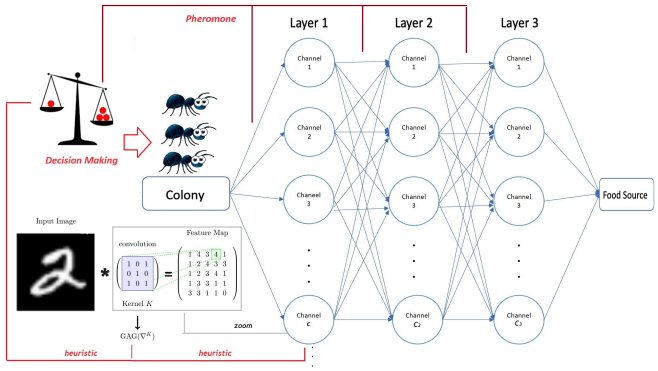

Figure 3: Representation of the graph $\mathcal{G}$ generated by the ACO-P algorithm, encompassing 3 layers; each layer i is composed of C_i channels (nodes representing DNN components); a particular channel c in the first layer, representing a kernel in a CNN model, is zoomed in to show its details.

Algorithm 1 describes the main steps regarding training-pruning procedures. In step 1, the algorithm starts by generating the graph $\mathcal{G}$ with the same dimensions as the original NN structure, ie., the number of nodes (channels) in each layer of $\mathcal{G}$ corresponds to the number of components suitable to be pruned in each NN layer. In the case of CNN models, each component represents a kernel in the neural model. For each independent run of ACO-P to a particular DNN model (loop from step 2 to 35), MEM (the set of subgraphs $\{\text{sub}\mathcal{G}^1, \ldots, \text{sub}\mathcal{G}^R\} \mid \text{sub}\mathcal{G}^r \in \mathcal{G}$ whose nodes will be pruned along the entire learning-pruning process) is initialized in step 3.

Subsequently, the weighted edges $\mathbf{w}_{\mathcal{G}}$ connecting the channels are initialized in step 4 with unitary values and weights $\mathbf{w}_{NN}$ within the NN model are randomly initialized in step 5. It is essential to point out that the concept of weights in the ACO-P graph differs from the concept of NN weights. The latter ($\mathbf{w}_{NN}$) represent elements of neural models which are updated based on gradient descent rules (e.g. convolutional kernels in CNN models, individual weights in the fully connected layers or head attention parameters in Transformers), whereas weights $\mathbf{w}_{\mathcal{G}}$ of graph $\mathcal{G}$ edges connect nodes (channels) and have their values updated based on two procedures: pheromone deposit and pheromone evaporation.

After $\mathcal{G}$ and NN weights initialization, steps 6 to 33 concerning the NN training loop with E epochs start. They encompass forward and backward phases (steps 9 and 10, respectively), in which information regarding every sample s, present in the current batch $\mathcal{B}$, flows from NN input until its output and the error of s, which is

Algorithm 1 ACO-P

1: Initialize the structure of graph $\mathcal{G}$ based on NN dimension
2: **for** each entire and independent learning process **do**
3: Initialize $MEM = \emptyset$, memory storing pruned subgraphs
4: Initialize $\mathcal{G}$ edge weights ($\mathbf{w}_{\mathcal{G}}$) with unitary values
5: Randomly initialize weights ($\mathbf{w}_{NN}$) ▹ Pytorch default: Kaiming uniform initialization [21]
6: **for** *epochs* from 1 to E **do**
7: **for** each batch $\mathcal{B}$ in the training dataset **do**
8: **for** each sample s in batch $\mathcal{B}$ **do**
9: $Foward(s, NN)$
10: $Backward(error(s), NN)$
11: calculate $\nabla^c(s), \forall c$ ▹ gradient of component c for sample $s \in \mathcal{B}$
12: **end for**
13: $\Delta \mathbf{w}_{NN} \propto (\nabla^c = \sum_{s \in \mathcal{B}} \nabla^c(s))$ ▹ Update, guided by opt, NN weights (e.g. based on Equation 1)
14: **for** each channel c in graph $\mathcal{G}$ **do**
15: $Heur_c \leftarrow \max(GAG(\nabla^c))$ ▹ Heuristic
16: **end for**
17: **if** $\text{mod}(epoch, E_p) = 0$ **then**
18: **for** round r from 1 to R **do**
19: **for** ACO iteration from 1 to MAX_{it} **do**
20: every ant $a \in \mathcal{A}$ makes a random choice to construct paths (e.g. based on Equation 2)
21: every ant $a \in \mathcal{A}$ updates pheromone in paths whose edges have weights $\mathbf{w}_{\mathcal{G}}$ (e.g. based on Equation 3)
22: **end for**
23: $\text{sub}\mathcal{G}^r \leftarrow$ best path in $\mathcal{G}$
24: $MEM = MEM \bigcup \text{sub}\mathcal{G}^r$ ▹ updating the set $MEM = \{\text{sub}\mathcal{G}^1, \text{sub}\mathcal{G}^2, \ldots, \text{sub}\mathcal{G}^{r-1}\}$ with the new subgraph
25: **end for**
26: $prunComp \leftarrow \text{getChannels}(\{\text{sub}\mathcal{G}^1, \ldots, \text{sub}\mathcal{G}^R\})$
27: $recovComp \leftarrow \text{RecovChannels}(rec * E_p, MEM)$
28: $MEM = MEM' \mid \text{sub}\mathcal{G}^r \notin MEM'$
29: Update NN structure with pruned/recov comp.
30: Reset ACO's *pheromones* for next batch cycle
31: **end if**
32: **end for** ▹ ——————————————————batch
33: **end for** ▹ ——————————————————epochs
34: output: final NN structure and weights
35: **end for** ▹ ——————————————————learning process

the basis of loss function $\mathcal{L}$, is backpropagated. Thus, the gradient for the sth sample ($\nabla^c(s)$), the one associated with component c, can be calculated in step 11. When the loop of the current batch $\mathcal{B}$ finishes (steps 8 to 12), the cumulative gradient of component c for all samples in $\mathcal{B}$, given by $\nabla^c = \sum_{s \in \mathcal{B}} \nabla^c(s)$, can be updated in step 13.

Considering the CNN model depicted in Figure 1, whose component (kernel) is highlighted in Figure 3, the gradient of component c corresponds to the gradient of kernel K. Therefore, as soon as calculations performed for all samples in batch $\mathcal{B}$ finish, the component weights (for example the kernel weights) are updated in step 13 proportionally to the cumulative gradient (∇^c), according to Equation 1 (e.g, for mBGD optimization algorithm).

$$\mathbf{w}_{NN}(t+1) = \mathbf{w}_{NN}(t) - \eta \nabla^c \tag{1}$$

In step 15, the algorithm calculates the heuristic of each channel in $\mathcal{G}$. See Equation 2 for more details.

At every set of E_p epochs (E_p represents the pruning frequency) the ACO algorithm is executed R times (steps 18 to 25) aiming to select the channels to be pruned at the current batch $\mathcal{B}$. In each round, $r = 1, ..., R$, it evolves for MAX_{it} iterations, and the best path (ie. the subgraph sub$\mathcal{G}^r$ with the highest $\sum_c \nabla^c$) is included in the set of paths whose channels are pruned along the training. The steps performed along MAX_{it} by the selection rule of the proposed approach (19 to 22), can be those of traditional ACO [35].

For traditional ACO algorithms, whenever an ant arrives at a specific node c in layer i of graph $\mathcal{G}$, it selects, among those available in the next layer $i+1$, a node d to be included in its path according to the probability:

$$p(c,d) = \frac{[\tau(c,d)]^\alpha \cdot [\eta(c,d)]^\beta}{\sum [\tau(c,d)]^\alpha \cdot [\eta(c,d)]^\beta} \tag{2}$$

where $p(c,d)$ is the probability of following the edge (c,d); variable $\tau(c,d)$ denotes the amount of pheromone and $\eta(c,d)$ denotes the heuristic value, both associated with an edge connecting channels c and d, with $\alpha \in [0,1]$ indicating the relative importance of pheromone and $\beta \in [0,1]$ indicating the relative relevance of heuristic information.

When ants complete their solution by arriving at the Food Source node, pheromone trails can be updated, increasing or decreasing the trail levels corresponding to moves that are part of "good quality" or "bad quality" solutions, respectively. The update rule of ACO pheromone is defined by Equation 3.

$$\tau(c,d) \leftarrow (1-\rho) \cdot \tau(c,d) + \sum_{a \in \mathcal{A}} \Delta\tau_a(c,d) \tag{3}$$

where parameter ρ denotes the pheromone decay factor, $\mathcal{A}$ is the total of ants in ACO, and $\Delta\tau_a(c,d)$ is the amount of pheromone deposited by ant a and its calculation is based on Equation 4.

$$\Delta\tau_a(c,d) = \begin{cases} 1/C_a, & \text{if ant } a \text{ uses } (c,d) \text{ in its path} \\ 0, & \text{otherwise} \end{cases} \tag{4}$$

here C_a is the total cost of the tour of ant a, given by the sum of the heuristic values of its path.

The pheromone update rule outlined in Equations 3 and 4 is recognized as the Ant System. This choice has been made because the given example is straightforward, and the focus is not on the intricacies of evolutionary algorithms, but rather on new pruning strategy development. Consequently, this leaves ample room for further refinement and enhancement.

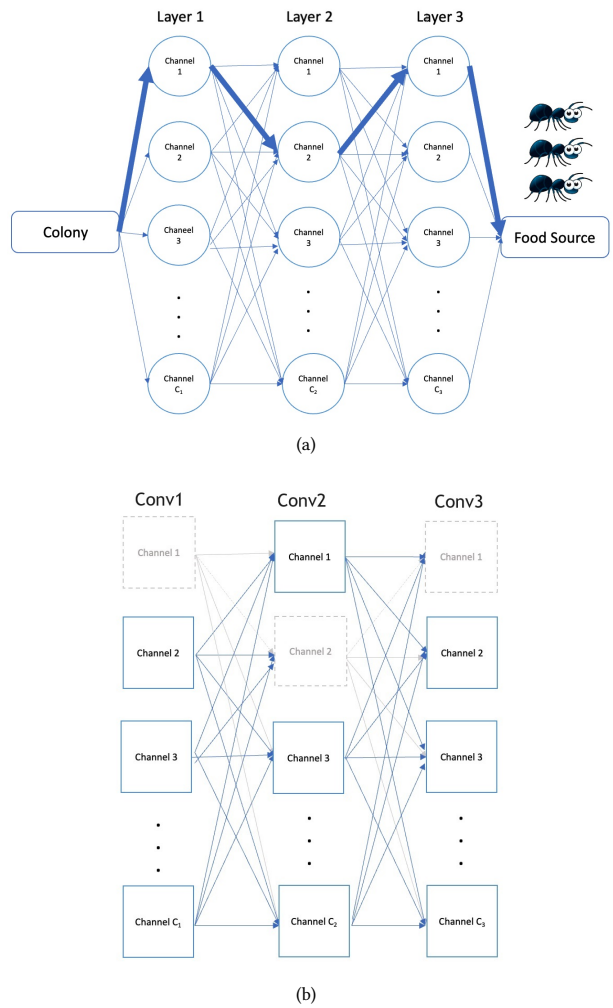

Figure 4: Final stage of $\mathcal{G}$ and NN structure: (a) the best subgraph of a particular round is highlighted, it represents the best path between the Colony and the Food Source; (b) NN structure without the channels of the selected subgraph indicating that every kernel and feature map associated with those channels have been pruned.

At the end of MAX_{it} iterations, ACO-P proceeds by selecting, in step 23, the best subGraph (sub$\mathcal{G}^r$) to have its components pruned. Subsequently, in step 24, the set of subGraphs to be pruned is updated with the new best-found subgraph (sub$\mathcal{G}^r$).

Figure 4(a) illustrates the final stage (after evolving for MAX_{it} iterations) of a single round of the ACO algorithm, when it selects the best subgraph to be pruned in that round. Figure 4(b) shows the CNN structure after the channel elimination in the selected subgraph.

In ACO-P, after R rounds of the loop from step 18 to 25, all the channels that compose the selected subgraphs, ie., channels from the set {sub$\mathcal{G}^1$, sub$\mathcal{G}^2$, ..., sub$\mathcal{G}^R$}, are expected to have their associated components pruned (in step 26) from the NN structure unless they have already been pruned in previous batches or epochs.

ACO-P also allows recovering pruned channels in step 27. If a channel has been pruned once but has not been chosen again along $rec * E_p$ epochs, its corresponding component returns to the NN structure with the weight values it had when it was pruned for the first time. Then, in step 28, the MEM set has such a subGraph eliminated from the list of already eliminated subgraphs.

Finally, ACO-P performs the final steps (29 and 30). First, the algorithm prunes the NN structure based on the current Batch $\mathcal{B}$ and its associated cumulative gradient information (∇^c) and then it resets ACO information for the next batch.

As an important contribution of the proposed approach, ACO-P calculates the heuristic of each channel in $\mathcal{G}$ through an innovative mechanism proposed in this paper for CNN models: the Global Average Gradient (GAG). Suppose the channel is located at the first layer. In that case, it receives a unique 2D input (the black and white image in our case), and GAG performs the global average pooling operation over the gradient of the component (the kernel in our case, as depicted in Figure 3).

However, if the channel is located at the second layer of $\mathcal{G}$ or more, it receives more than one 2D input. Then, ACO-P performs GAG depending on the particular characteristics of the addressed deep model. Figure 5 shows that in CNN models, the channel receives, from the previous layer, the multidimensional kernel output.

According to Figure 5, in the case of CNN models, each kernel in layer i receives multiple 2D inputs from layer $i-1$ and produces more than one output (feature maps). During the backward phase, each feature map provides a corresponding gradient map with the same dimension as the associated kernel. Then, the GAG of each gradient map, is calculated and a vector with aggregated values (one for each output gradient map) is created giving rise (through the max operator) to the heuristic of the respective channel.

Unlike the traditional Global Average Pooling (GAP), which directly depends on the feature map elements, GAG is calculated based on the gradient resulting from each feature map obtained from the backpropagation of the sample errors during the training. At every iteration, the global average value of each gradient map is calculated and the max value of the resulting array is updated as the heuristic value of the corresponding channel. Kernels with higher gradients have a higher chance of being pruned with GAG. Although the proposed heuristic described here is based on kernel components of a CNN model, the idea behind GAG is general. It can be extended to different deep models whose components are updated by gradient descent.

4 EXPERIMENTS AND RESULTS

Aiming to validate the proposed approach (ACO-P), we consider a CNN model to address a classification problem using the MNIST dataset. It encompasses a collection of binary images of handwritten digits commonly used to train machine learning and pattern recognition methods.

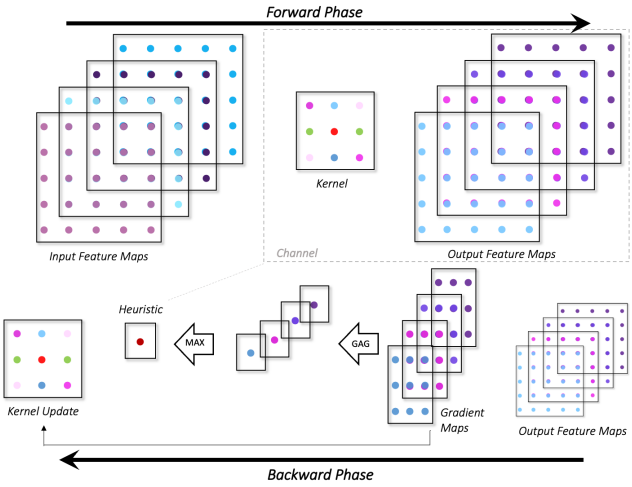

Figure 5: Representation of Global Average Gradient (GAG) of a multidimensional Kernel in a CNN model. At the top, we have the forward phase and the resulting feature maps. At the bottom, we have the backward phase, when the kernel is updated based on the gradient maps, which also serve as the basis for the GAG and heuristic calculations.

According to [26], the MNIST dataset comprises a training set of 60,000 handwritten digit images and a test set of 10,000 images. The training set consists of patterns from approximately 250 different writers. The dataset includes ten classes representing the digits from 0 to 9, making it a well-established benchmark in computer vision research.

In the experiments, the CNN structure used to test ACO-P and to compare approaches is a three-convolutional layer CNN as depicted in Figure 1. We evaluate the performance of the considered approaches by running different initial CNN weights (each run relates to the loop from step 2 to 35). Table 1 shows the parameters used for the CNN training, ACO algorithm, and those considered in both parts of our proposal.

All models were developed and trained using the PyTorch framework [36] (version 2.2.1). In this study, we conducted experiments on Google Colab Pro [15], utilizing its back-end GPU on Google Compute Engine in Python 3 with 12.7 GB of system RAM, 15.0 GB of GPU RAM, and 201.2 GB of disk space. We used the same instance type, minimized background processes, and averaged the results over 10 runs, to obtain reliable performance metrics. Furthermore, we used the `time.perf_counter()` function, which ensures that the measured training duration is not affected by any changes in the system clock, providing a reliable and consistent metric for performance evaluation.

Table 2 shows training and testing average accuracy, training time, and pruning rate for four approaches, including ACO-P. One approach (No-pruning) does not consider pruning. Two other pruning approaches (Random-P and RNN-P) consider a random pruning with the same frequency E_p of ACO-P, and the RNN pruning proposed in [28], respectively.

Table 1: Parameters of the proposed approach: CNN training and ACO algorithm

Name/symbol (CNN)	value	Description
Optimizer Opt	mBGD	In mini-batch Gradient Descent, instead of using the entire dataset for each epoch, only a small batch is selected to calculate the gradient and update the parameters;
learning rate η	0.01	controls the size of each weight update step during the training;
loss function $\mathcal{L}$	MSE	Mean squared error (measures the error to be backpropagated);
activation function σ	ReLU	the rectified linear unit, a non linear function applied to each feature map element;
batch size $\lvert\mathcal{B}\rvert$	100	refers to the number of training samples propagated through the network before each CNN weight update takes place;
Max epochs E	20	stop criterion of every learning process: the number of times each sample in the entire dataset is trained;
Total learning runs	10	the number of times the CNN learning process (with E epochs) is repeated using different initial CNN weights in each one.
Name/Symbol (ACO)	**value**	**Description**
pheromone relevance α	0.7	relevance of pheromone information in the probabilistic decision;
heuristic relevance β	0.3	relevance of heuristic information in the probabilistic decision;
evaporation rate ρ	0.1	rate of pheromone evaporation in the update equation;
total of ants $\mathcal{A}$	20	total of ants in every ACO round r;
prune at epoch E_p	2	frequency of ACO-based pruning along the CNN training process;
limit for recovering rec	3	max of E_p epochs a channel can stay pruned before being recovered;
max ACO it MAX_{it}	50	Maximum number of iterations per ACO round;
max ACO rounds R	5	the number of times the ACO algorithm is executed (with MAX_{it} iterations) for the current batch $\mathcal{B}$.
Name/Symbol (Both)	**value**	**Description**
convolutional layers L	3	total of convolutional layers in the complete CNN structure;
kernels/layer C_1	16	number of kernels in conv. layer 1 of the complete CNN structure which is also the number of channels in the 1st graph layer;
kernels/layer C_2	32	number of kernels in conv. layer 2 of the complete CNN structure which is also the number of channels in the 2nd graph layer;
kernels/layer C_3	32	number of kernels in conv. layer 3 of the complete CNN structure which is also the number of channels in the 3rd graph layer;

Table 2: Average accuracy AvACC (%) of training and testing, time for training (s), and pruning rate (%) with standard deviation as subscripts.

Approach	Training (AvACC%)	Testing (AvACC%)	Time for Training (s)	Pruning Rate (%)
No pruning	$99.60_{0.001}$	$99.01_{0.001}$	$235.71_{3.17}$	-
Random-P	$57.31_{0.36}$	$58.18_{0.37}$	$271.23_{1.75}$	$60_{0.40}$
RNN-P	$93.14_{0.003}$	$93.42_{0.008}$	$683.4_{20.226}$	$14_{0.039}$
ACO-P	$\mathbf{94.53}_{0.013}$	$\mathbf{95.48}_{0.013}$	$245.4_{1.273}$	$\mathbf{55}_{0.074}$

The results of Table 2 are computed for 10 runs of each approach with 20 epochs each. According to Table 2, Random-P provides the worst training and testing results and pruning rate. RNN-P presents no relevant difference between training and testing, and ACO-P achieves slightly better training and testing results in comparison to RNN-P. However, a significant difference between ACO-P and RNN-P pruning rates can be observed in Table 2. It is important to highlight that RNN-P disregards the first convolutional layer and uses an extra encoding layer attached to the convolutional layers. Therefore, it increases the number of weights compared to our approach (ACO-P), which contributes for the low pruning rate of 14%. Moreover, the ACO-P algorithm performs the training process twice as fast as the RNN Pruning.

As previously discussed, the proposed algorithm calculates the heuristic of each channel in $\mathcal{G}$ through the innovative mechanism called - Global Average Gradient (GAG). This method is specifically designed for the ACO algorithm, enhancing its decision-making process by providing a more nuanced understanding of each channel's contribution to the overall model's performance. Initially, the proposed heuristic involved utilizing the CNN weights as the heuristic value and implementing the global average pooling method for the calculation. Unfortunately, this method did not achieve the desired outcomes, with training and testing accuracy lower than 80%,

highlighting the significance of choosing an effective heuristic. A novel approach was then introduced by employing the gradients of weights as the heuristic value. This innovative shift seeks to exploit dynamic and informative characteristics of weight gradients.

Aiming to evaluate comparison approach scalabilities, different CNN architectures have been tested according to Table 3. It shows architectures from less (CNN1) to more (CNN3) kernels within each convolutional layer than the addressed model in bold (CNN2). Table 3 also includes the average testing accuracy of each architecture across different approaches, with the standard deviation provided in subscripts.

Table 3: CNN architectures, kernel sizes and testing accuracy

CNN	kernel per layer	No-Pruning (AvACC%)	ACO-P (AvACC%)	RNN-P (AvACC%)
CNN 1	$8 \times 16 \times 16$	$98.76_{0,001}$	$95.29_{0.025}$	$90.52_{0.013}$
CNN 2	$\mathbf{16 \times 32 \times 32}$	$99.01_{0.001}$	$\mathbf{95.48}_{0.013}$	$93.42_{0.001}$
CNN 3	$16 \times 32 \times 32 \times 64$	$99.07_{0.001}$	$98.11_{0.005}$	$97.92_{0.001}$

The training time results are shown in Figure 6. Notably, the ACO-P approach performs comparable to the No-Pruning strategy, despite the additional cost of ACO running within the training procedure. Conversely, the RNN Prunning exhibits a substantially higher training time, signifying an intense computational demand. The scalability of RNN-P is very poor compared to ACO-P. It could be challenging for huge-size CNNs.

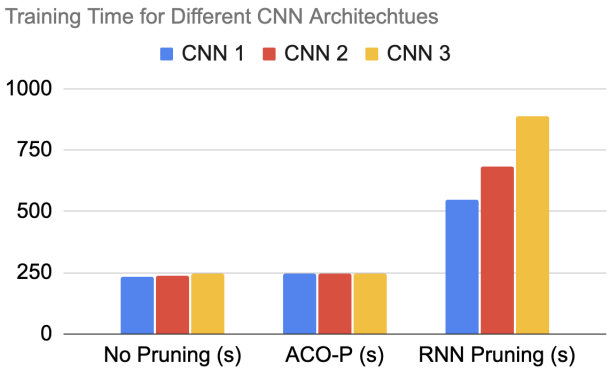

Figure 6: Training time (s) for CNN models with less (CNN1) and more (CNN3) kernels than the addressed model (CNN2) for No-pruning, ACO-P, and RNN-P.

5 CONCLUSION

Deep Neural Networks work well in several tasks mainly due to their multiple levels of computational operations (that depend on a large set of parameters). DNNs are useful for analyzing vast volumes of data, but improving the network's effectiveness typically requires large and deep models, which raises the computational cost and makes their use difficult in many applications.

Pruning algorithms are necessary strategies to mitigate the computational training cost of DNNs. Dynamic pruning provides better approaches toward more adaptable pruned networks. The Ant Colony Optimization Pruning (ACO-P) strategy proposed in the present paper offers a brand-new technique based on swarm intelligence that relieves the need for a secondary neural network to make pruning decisions. Moreover, we provided an innovative solution that uses the global average pooling of gradient maps to guide ants toward more suitable channels for pruning.

We tested the proposal on a CNN model used to solve the classification problem of MNIST dataset. Upon a comparative analysis, the algorithm proposed herein (ACO-P), demonstrated competitive performance relative to both random pruning and RNN-P, a well-established pruning algorithm employing reinforcement learning and a recurrent neural network approach. Concerning the learning process without pruning, ACO-P achieved the most competitive performance with only 3.5% of accuracy reduction compared to 5.6% reduction of RNN Pruning and almost 41% of the random approach. Furthermore, ACO-P achieved an approximately fourfold increase in pruning rate on average compared to RNN-P, which employs Q-learning and a secondary recurrent neural network. Additionally, our implementation of ACO-P reduces the computational burden of the training-pruning process by nearly threefold compared to RNN-P.

The Ant System that supports our pruning approach, served as an accessible introduction to the concepts involved, laying a foundation that accepts further exploration of more sophisticated and advanced methodologies. This approach highlights the potential for substantial enhancements in future development cycles, as it establishes the fundamental principles necessary for further advancement.

Many pruning methods of the literature focus on static network models while dynamic pruning is recent. It is the first time a dynamic pruning strategy based on ACO has been proposed, to the best of our knowledge. However, some limitations of the results should be mentioned. A single dataset for the experiments may limit the results on how the pruning reduction rates are affected by more complex datasets.

Although the results for training and pruning rates are good, more experiments should be done to clarify the issues of scalability and inference implementation. Because newer DNN models were not considered, potential research topics should include transformers, graph neural networks, and diffusion models. Further work should also consider explanations provided by ACO-P to improve interpretability. Moreover, the proposal can be extended to other applications, and compared with other evolutionary pruning strategies. Further assessments need to be done to evaluate all the proposed components of the algorithm, such as the ACO pheromone update rule and the heuristic value.

ACKNOWLEDGMENTS

The authors thank Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES). MD thanks Conselho Nacional de Desenvolvimento Científico e Tecnológico - CNPq (process 313122/2023-7). RD thanks Serviço Nacional de Aprendizagem Industrial - Brasil (SENAI).

REFERENCES

[1] Mohamed Abd Elaziz, Abdelghani Dahou, Laith Abualigah, Liyang Yu, Mohammad Alshinwan, Ahmad M Khasawneh, and Songfeng Lu. 2021. Advanced metaheuristic optimization techniques in applications of deep neural networks: a review. *Neural Computing and Applications* (2021), 1–21.

[2] Ossama Abdel-Hamid, Abdel-rahman Mohamed, Hui Jiang, Li Deng, Gerald Penn, and Dong Yu. 2014. Convolutional neural networks for speech recognition. *IEEE/ACM Transactions on audio, speech, and language processing* 22, 10 (2014), 1533–1545.

[3] Rand Jawad Kadhim Almahmood and Adem Tekerek. 2022. Issues and Solutions in Deep Learning-Enabled Recommendation Systems within the E-Commerce Field. *Applied Sciences* 12, 21 (2022), 11256.

[4] EO Arkhangelskaya and Sergei Igorevich Nikolenko. 2023. Deep learning for natural language processing: a survey. *Journal of Mathematical Sciences* (2023), 1–50.

[5] Edvinas Byla and Wei Pang. 2019. Deepswarm: Optimising convolutional neural networks using swarm intelligence. In *UK Workshop on Computational Intelligence*. Springer, 119–130.

[6] Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. 2023. A survey on deep neural network pruning-taxonomy, comparison, analysis, and recommendations. *arXiv preprint arXiv:2308.06767* (2023).

[7] Shaveta Dargan, Munish Kumar, Maruthi Rohit Ayyagari, and Gulshan Kumar. 2020. A survey of deep learning and its applications: a new paradigm to machine learning. *Archives of Computational Methods in Engineering* 27 (2020), 1071–1092.

[8] Tiancan Deng. 2022. A survey of convolutional neural networks for image classification: Models and datasets. In *2022 International Conference on Big Data, Information and Computer Network (BDICN)*. IEEE, 746–749.

[9] Travis Desell, Sophine Clachar, James Higgins, and Brandon Wild. 2015. Evolving deep recurrent neural networks using ant colony optimization. In *Evolutionary Computation in Combinatorial Optimization: 15th European Conference, EvoCOP 2015, Copenhagen, Denmark, April 8-10, 2015, Proceedings 15*. Springer, 86–98.

[10] Shi Dong, Ping Wang, and Khushnood Abbas. 2021. A survey on deep learning and its applications. *Computer Science Review* 40 (2021), 100379.

[11] Marco Dorigo. 1992. Optimization, learning and natural algorithms. *Ph. D. Thesis, Politecnico di Milano* (1992).

[12] Jan Egger, Christina Gsaxner, Antonio Pepe, Kelsey L Pomykala, Frederic Jonske, Manuel Kurz, Jianning Li, and Jens Kleesiek. 2022. Medical deep learning—A systematic meta-review. *Computer methods and programs in biomedicine* 221 (2022), 106874.

[13] AbdElRahman A ElSaid, Alexander G Ororbia, and Travis J Desell. 2019. The ant swarm neuro-evolution procedure for optimizing recurrent networks. *CoRR abs/1909.11849* (2019).

[14] Mehdi Gheisari, Fereshteh Ebrahimzadeh, Mohamadtaghi Rahimi, Mahdieh Moazzamigodarzi, Yang Liu, Pijush Kanti Dutta Pramanik, Mohammad Ali Heravi, Abolfazl Mehbodniya, Mustafa Ghaderzadeh, Mohammad Reza Feylizadeh, et al. 2023. Deep learning: Applications, architectures, models, tools, and frameworks: A comprehensive survey. *CAAI Transactions on Intelligence Technology* 8, 3 (2023), 581–606.

[15] Google. [n. d.]. Google Colab Pro. https://colab.research.google.com. Accessed on: March, 2024.

[16] Tasneem Gorach. 2018. Deep convolutional neural networks-a review. *Int Res J Eng Tech (IRJET)* 5, 07 (2018).

[17] Song Han, Huizi Mao, and William J Dally. 2016. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*. (2016).

[18] Song Han, Jeff Pool, John Tran, and William Dally. 2015. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems* 28 (2015).

[19] Awni Hannun, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen, Ryan Prenger, Sanjeev Satheesh, Shubho Sengupta, Adam Coates, and Andrew Ng. 2014. Deep speech: Scaling up end-to-end speech recognition. *CoRR abs/1412.5567* (2014).

[20] Stephen Hanson and Lorien Pratt. 1988. Comparing biases for minimal network construction with back-propagation. *Advances in neural information processing systems* 1 (1988), 177–185.

[21] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*. 1026–1034.

[22] David A. Huffman. 1952. A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE* 40, 9 (1952), 1098–1101. https://doi.org/10.1109/JRPROC.1952.273898

[23] Saidul Islam, Hanae Elmekki, Ahmed Elsebai, Jamal Bentahar, Nagat Drawel, Gaith Rjoub, and Witold Pedrycz. 2023. A comprehensive survey on applications of transformers for deep learning tasks. *Expert Systems with Applications* (2023), 122666.

[24] Athanasios Karapantelakis, Pegah Alizadeh, Abdulrahman Alabassi, Kaushik Dey, and Alexandros Nikou. 2024. Generative ai in mobile networks: a survey. *Annals of Telecommunications* 79, 1 (2024), 15–33.

[25] Asifullah Khan, Anabia Sohail, Umme Zahoora, and Aqsa Saeed Qureshi. 2020. A survey of the recent architectures of deep convolutional neural networks. *Artificial intelligence review* 53, 8 (2020), 5455–5516.

[26] Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. [n. d.]. The MNIST Database of handwritten digits. http://yann.lecun.com/exdb/mnist/. Access: 2023-07-09.

[27] Tailin Liang, John Glossner, Lei Wang, Shaobo Shi, and Xiaotong Zhang. 2021. Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing* 461 (2021), 370–403.

[28] Ji Lin, Yongming Rao, Jiwen Lu, and Jie Zhou. 2017. Runtime neural pruning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 2178–2188.

[29] Min Lin, Qiang Chen, and Shuicheng Yan. 2014. Network in network. *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, 14-16* (2014).

[30] Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. 2022. A survey of transformers. *AI Open* (2022).

[31] Meng Liu, Hongyang Gao, and Shuiwang Ji. 2020. Towards deeper graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 338–348.

[32] Qing Liu, Ningyu Zhang, Wenzhu Yang, Wang Sile, Zhenchao Cui, Xiangyang Chen, and Liping Chen. 2017. A Review of Image Recognition with Deep Convolutional Neural Network. In *International Conference on Intelligent Computing*. 69–80.

[33] Wenping Ma, Xiaobo Zhou, Hao Zhu, Longwei Li, and Licheng Jiao. 2021. A two-stage hybrid ant colony optimization for high-dimensional feature selection. *Pattern Recognition* 116 (2021), 107933.

[34] Amina N Muhammad, Ali M Aseere, Haruna Chiroma, Habib Shah, Abdulsalam Y Gital, and Ibrahim Abaker Targio Hashem. 2021. Deep learning application in smart cities: recent development, taxonomy, challenges and research prospects. *Neural computing and applications* 33 (2021), 2973–3009.

[35] Simon Parsons. 2005. Ant Colony Optimization by Marco Dorigo and Thomas Stützle. *The Knowledge Engineering Review* 20, 1 (2005), 92–93.

[36] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*. 8024–8035.

[37] Yongming Rao, Jiwen Lu, Ji Lin, and Jie Zhou. 2018. Runtime network routing for efficient image classification. *IEEE transactions on pattern analysis and machine intelligence* 41, 10 (2018), 2291–2304.

[38] Khalid Salama and Ashraf M Abdelbar. 2014. A novel ant colony algorithm for building neural network topologies. In *International Conference on Swarm Intelligence*. Springer, 1–12.

[39] Koosha Sharifani and Mahyar Amini. 2023. Machine Learning and Deep Learning: A Review of Methods and Applications. *World Information Technology and Engineering Journal* 10, 07 (2023), 3897–3904.

[40] Xiaoxuan Shen, Baolin Yi, Zhaoli Zhang, Jiangbo Shu, and Hai Liu. 2016. Automatic recommendation technology for learning resources with convolutional neural network. In *2016 international symposium on educational technology (ISET)*. IEEE, 30–34.

[41] Jiankun Sun, Xiong Luo, Honghao Gao, Weiping Wang, Yang Gao, and Xi Yang. 2020. Categorizing malware via a word2vec-based temporal convolutional network scheme. *Journal of Cloud Computing* 9, 1 (2020), 1–14.

[42] Pete Warden and Daniel Situnayake. 2019. *Tinyml: Machine learning with tensorflow lite on arduino and ultra-low-power microcontrollers*. O'Reilly Media.

[43] Sheng Xu, Anran Huang, Lei Chen, and Baochang Zhang. 2020. Convolutional Neural Network Pruning: A Survey. In *2020 39th Chinese Control Conference (CCC)*. IEEE, 7458–7463.

[44] Ling Yang, Zhilong Zhang, Yang Song, Shenda Hong, Runsheng Xu, Yue Zhao, Wentao Zhang, Bin Cui, and Ming-Hsuan Yang. 2023. Diffusion models: A comprehensive survey of methods and applications. *Comput. Surveys* 56, 4 (2023), 1–39.

[45] Zhaolin Yuan, Jinlong Hu, Di Wu, and Xiaojuan Ban. 2020. A dual-attention recurrent neural network method for deep cone thickener underflow concentration prediction. *Sensors* 20, 5 (2020), 1260.

[46] Matthew D Zeiler and Rob Fergus. 2014. Visualizing and understanding convolutional networks. In *European conference on computer vision*. Springer, 818–833.